

# Page "Mon Stock" — Retards Backend

**Date :** 2026-05-16 **Commit backend :** 570f074 **Page :** /stocks , /stocks/nouveau , /stocks/:id/modifier , /stocks/configuration

---

## Résumé

Le frontend stock est entièrement conforme aux standards CLAUDE.md. Tous les endpoints existants sont connectés et testés (12 tests, 6 hooks). Les problèmes restants sont tous côté backend — des champs et endpoints manquants qui empêchent certaines fonctionnalités UI de fonctionner.

---

## 1. Catégorie d'ingrédient — INEXISTANT

**Impact :** les filtres "Toutes catégories" ne fonctionnent pas, tous les produits affichent la même catégorie.

**Modèle actuel** ( apps/inventory/models.py ) :

```
class Ingredient(models.Model):
    nom = models.CharField(max_length=100)
    unite = models.CharField(max_length=50)
    prix_unitaire = models.DecimalField(...)
    # ← PAS de champ catégorie
```

**Ce qu'il faut ajouter :**

```
class CategorieIngredient(models.Model):
    nom = models.CharField(max_length=100)
    class Meta:
        db_table = "T_HOLLY_PI_CATEGORIES_INGREDIENTS"

class Ingredient(models.Model):
    # ... champs existants ...
    categorie = models.ForeignKey(
        CategorieIngredient,
        on_delete=models.SET_NULL,
        null=True, blank=True,
    )
```

**Endpoints à créer :**

- GET /api/ingredient-categories/ — lister
- POST /api/ingredient-categories/ — créer
- Ajouter categorie\_id dans le serializer Ingredient
- Ajouter ingredient\_category (ou categorie\_\_nom ) dans le serializer Stock

**Workaround frontend actuel :** category: "epicerie" hardcodé pour tous les produits.

---

## 2. Zone de stockage — INEXISTANT

**Impact** : tous les produits tombent dans "Réserve sèche". La page `/stocks/configuration` (CRUD zones) ne persiste rien — perdu au refresh.

**Modèle actuel** ( `apps/inventory/models.py` ) :

```
class Stock(models.Model):
    restaurant = models.ForeignKey(Restaurant, ...)
    ingredient = models.ForeignKey(Ingredient, ...)
    quantite_en_stock = ...
    seuil_alerte = ...
    cout_moyen_pondere = ...
    # ← PAS de zone de stockage
```

**Ce qu'il faut ajouter** :

```
class ZoneStockage(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE)
    nom = models.CharField(max_length=100)
    description = models.TextField(blank=True)
    class Meta:
        db_table = "T_HOLLY_PI_ZONES_STOCKAGE"
        unique_together = ('restaurant', 'nom')

class Stock(models.Model):
    # ... champs existants ...
    zone = models.ForeignKey(
        ZoneStockage,
        on_delete=models.SET_NULL,
        null=True, blank=True,
    )
```

**Endpoints à créer** :

- GET `/api/stock-zones/?restaurant_id=X` — lister les zones d'un restaurant
- POST `/api/stock-zones/` — créer une zone
- PATCH `/api/stock-zones/{id}/` — renommer
- DELETE `/api/stock-zones/{id}/` — supprimer
- Ajouter `zone_id` dans `PatchedStockRequest` pour assigner un stock à une zone

**Workaround frontend actuel** : `storageZone: "reserve_seche"` hardcodé + constantes JS `DEFAULT_STORAGE_ZONES` .

---

### 3. Lien Stock ↔ Fournisseur — INEXISTANT

**Impact** : `product.supplierId` est toujours `""` . Les filtres fournisseur sur la page stock ne fonctionnent pas. Le modal détail produit ne peut pas afficher le fournisseur.

**Situation actuelle** : aucun lien entre `Ingredient / Stock` et `Fournisseur` . Le `Reapprovisionnement` n'a pas non plus de FK fournisseur (seulement `restaurant` , `ingredient` , `quantite` , `prix_achat` ).

**Ce qu'il faut ajouter** (option la plus simple) :

```
class Ingredient(models.Model):
    # ... champs existants ...
    fournisseur = models.ForeignKey(
        'suppliers.Fournisseur',
        on_delete=models.SET_NULL,
        null=True, blank=True,
        help_text="Fournisseur principal de cet ingrédient",
    )
```

Puis exposer `fournisseur_id` et `fournisseur_name` dans le serializer `Stock`.

**Workaround frontend actuel :** `supplierId: ""` hardcodé.

---

## 4. Lignes de commande fournisseur — NON SUPPORTÉ

**Impact :** quand un gérant crée une commande fournisseur depuis la page stock (bouton "Commander"), la commande est créée **vide** — sans les produits sélectionnés.

**Endpoint actuel :** `POST /api/suppliers/orders/` accepte :

```
{
  "fournisseur_id": 1,
  "restaurant_id": 1,
  "notes": "...",
}
```

**Ce qu'il faut :** accepter un tableau `items` dans le body :

```
{
  "fournisseur_id": 1,
  "restaurant_id": 1,
  "notes": "...",
  "items": [
    { "ingredient_id": 5, "quantite": 10, "prix_unitaire": "3.80" },
    { "ingredient_id": 12, "quantite": 5, "prix_unitaire": "42.50" }
  ]
}
```

Ou créer un endpoint dédié `POST /api/suppliers/orders/{id}/lines/` pour ajouter des lignes après création.

**Workaround frontend actuel :** `data.items` est reçu du dialog mais jamais envoyé à l'API.

---

## 5. Catégories fournisseur — PRÊT, NON BRANCHÉ

**Impact :** mineur — l'endpoint existe ( `GET /api/suppliers/categories/` ) mais le frontend ne l'utilise pas encore.

**Pas de changement backend nécessaire.** Le frontend doit juste créer un hook `useSupplierCategories()` et l'afficher dans les filtres.

---

## 6. Champs `PATCH /stocks/{id}/` limités

**Situation** : le `PatchedStockRequest` n'accepte que `quantity_in_stock` et `alert_threshold`. Le nom, l'unité et le prix sont sur `Ingredient` (table séparée).

**Impact** : le formulaire d'édition stock ne peut modifier que la quantité et le seuil — le nom/unité/prix sont en lecture seule. C'est correct si c'est voulu (un ingrédient est partagé entre restaurants), mais si un gérant veut corriger un nom mal saisi, il ne peut pas.

**Option** : ajouter un `PATCH /api/ingredients/{id}/` (existe déjà dans le swagger — `PUT` dispo). Le frontend a déjà `useUpdateIngredient()`. Il faudrait juste que le formulaire d'édition stock propose aussi la modification de l'ingrédient sous-jacent si besoin.

---

## Priorisation

| Priorité | Tâche                                                                 | Effort                 |
|----------|-----------------------------------------------------------------------|------------------------|
| P0       | Lien Ingredient ↔ Fournisseur (FK + serializer)                       | ~2h                    |
| P0       | Lignes commande fournisseur (items dans POST orders)                  | ~3h                    |
| P1       | Catégorie ingrédient (modèle + CRUD + serializer)                     | ~3h                    |
| P1       | Zone de stockage (modèle + CRUD + FK sur Stock)                       | ~4h                    |
| P2       | Brancher catégories fournisseur au frontend                           | ~30min (frontend seul) |
| P2       | Permettre l'édition du nom/prix ingrédient depuis le formulaire stock | ~1h (frontend seul)    |